IA APLICADA AL DESARROLLO DE SOFTWARE

Optativa I: Fundamentos de la Inteligencia Artificial en la Ingeniería de Software

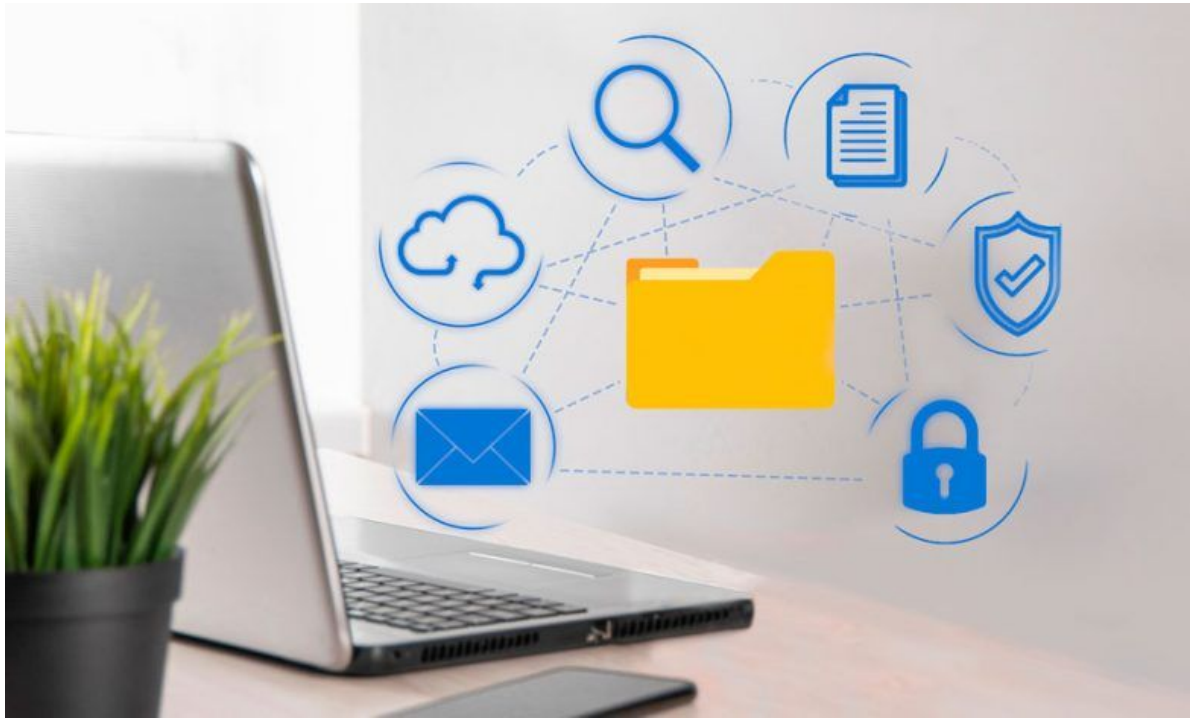


PARADIGMA ACTUAL

La Evolución:

2005-2015: Autocompletado estático (Sintaxis).
2015-2022: Sugerencias predictivas (Patrones).
2022-2024: Generación de código (LLMs básicos).
2025-Presente: Agentes Autónomos y Contexto Consciente (RAG, AST parsing, Comprensión del Negocio).

El Nuevo Rol: De Software Developer a Product Engineer / System Orchestrator.



IA APLICADA AL DESARROLLO DE SOFTWARE



Impacto en el Ciclo de Vida (SDLC):

- *Diseño*: Generación de diagramas UML/Mermaid y selección de stack tecnológico.
- *Implementación*: Síntesis de boilerplate, algoritmos y microservicios.
- *Testing*: Generación de casos de borde, fuzzing y pruebas de integración.
- *Mantenimiento*: Refactorización de código legado (Legacy Modernization).

No solo se trata de <<velocidad de entrega>>, sino *Reducción de Deuda Técnica* y *Disminución de Carga Cognitiva*.

CONCEPTOS CLAVE



- DESARROLLO ASISTIDO. Es la evolución tecnológica del desarrollo, donde un **programador** utiliza herramientas de Inteligencia Artificial para **automatizar, predecir y complementar sus tareas** de escritura de código.
- PAIR PROGRAMMING. Es una técnica de desarrollo ágil tradicional en la que **dos programadores humanos** trabajan juntos en una sola estación de trabajo (compartiendo pantalla, teclado y mouse)
- FLUJO HUMANO-IA. Es el **modelo operativo y metodológico** que redefine cómo interactúan los desarrolladores y las herramientas de IA. No se trata solo de usar la IA para "copiar y pegar" código rápido, sino de diseñar un proceso híbrido estructurado donde ambos aportan lo que mejor saben hacer.

DESARROLLO ASISTIDO POR IA (HERRAMIENTAS)

El Ecosistema de Herramientas (IDE-Native):

- *Editores Conscientes del Contexto*: Cursor, Windsurf, GitHub Copilot Enterprise.
- *Asistentes de Terminal*: CLI agents que entienden el estado del entorno.

Técnicas de Asistencia Avanzada:

- *Codebase Indexing*: La IA indexa el repositorio local para responder preguntas como "¿Dónde se implementa la lógica de autenticación de dos factores?".
- *Generación Dirigida por Pruebas* (TDD Asistido): La IA escribe los tests fallidos primero, y luego genera el código para pasarlos.
- *Explicación y Onboarding*: Comprensión instantánea de código de terceros o de otros equipos.











PAIR PROGRAMMING

El Compañero Sintético:

- Humano: Revisor de arquitectura, guardián de la lógica de negocio, auditor de seguridad.
- IA: Escritor de código veloz, explorador de alternativas, generador de documentación.

Dinámicas de Pair Programming con IA:

- El Navegador Sintético: Le pides a la IA que critique tu diseño antes de implementarlo.
- Refactorización en Vivo: "*Reescribe este bloque para que sea más legible y cumpla con los principios SOLID*".
- Caza de Bugs: "*Analiza este stack trace y sugiere 3 posibles causas raíz en la base de código*".



Nunca delegues el pensamiento crítico.

FLUJO DE TRABAJO HUMANO-IA (EL NUEVO SDLC)

Integración Continua / Despliegue Continuo (CI/CD):

- *Pre-Commit*: Agentes locales que validan seguridad, linting y tests antes de permitir el git push.
- *Code Review Automatizado:* Bots que revisan Pull Requests, detectan fugas de memoria, problemas de concurrencia y sugieren mejoras antes de que un humano pierda tiempo.
- *Remediación Autónoma:* Si un build falla en el pipeline, un agente de IA lee el log, identifica el error, crea un branch, corrige el código y abre un PR de solución.

Arquitectura Dirigida por IA:

- Uso de LLMs para simular cargas, predecir cuellos de botella y diseñar esquemas de bases de datos.



RETOS

Seguridad y Privacidad:

- Riesgo de Prompt Injection en agentes autónomos.
- Fugas de secretos (API Keys) en el contexto de la IA.

Propiedad Intelectual y Licencias:

- Contaminación de código (Copyleft) y auditoría de origen.

El Riesgo de la Homogeneización:

- Si todos usamos los mismos modelos, ¿estamos perdiendo la diversidad de enfoques algorítmicos?



RETOS

Seguridad y Privacidad:

- Riesgo de Prompt Injection en agentes autónomos.
- Fugas de secretos (API Keys) en el contexto de la IA.

Propiedad Intelectual y Licencias:

- Contaminación de código (Copyleft) y auditoría de origen.

El Riesgo de la Homogeneización:

- Si todos usamos los mismos modelos, ¿estamos perdiendo la diversidad de enfoques algorítmicos?



ASIGNACIÓN IVIRTUAL: Ingeniería de Software Potenciada por IA

- Aplicar metodologías de desarrollo asistido por Inteligencia Artificial, Pair Programming sintético y flujos de trabajo humano-IA para resolver un caso de uso de negocio, manteniendo el control de la arquitectura, la seguridad y la calidad del código.
- El objetivo no es que la IA escriba todo el código, sino que ustedes actúen como Arquitectos y Directores Técnicos (*Human-in-the-loop*).



Caso de Uso

Practica: "VetSync - Sistema de Gestión de Citas y Telemedicina Veterinaria"

Requerimientos de Negocio:

1. Registro de mascotas y dueños con historial médico.
2. Sistema de agendamiento de citas que evite choques de horarios para los veterinarios.
3. Módulo de notificaciones (simulado) por correo cuando la cita está próxima.
4. **Restricción Técnica:** Debe desarrollarse usando una arquitectura de microservicios (o modular monolito bien desacoplado), aplicando TDD (Desarrollo Guiado por Pruebas) y cumpliendo estrictamente con los principios SOLID.

FASE 1: Desarrollo Asistido por IA (Arquitectura y Contexto)

1. Diseño de Arquitectura:

Utilizar un LLM (Chat) para definir el esquema de base de datos y la estructura de carpetas.

Prompt sugerido:

a. "*Actúa como un Arquitecto de Software Senior. Para el caso de uso 'VetSync', propón un esquema de base de datos relacional (MySQL) y la estructura de directorios para un backend en [Node.js/Python/Java]. Justifica tus decisiones basándote en escalabilidad y principios SOLID.*"

FASE 1: Desarrollo Asistido por IA (Arquitectura y Contexto)

2. Configuración del Contexto del IDE:

Creen un archivo de reglas en la raíz de su proyecto (ej. .cursorrules o AGENTS.md) para dictar las convenciones de su equipo a la IA.

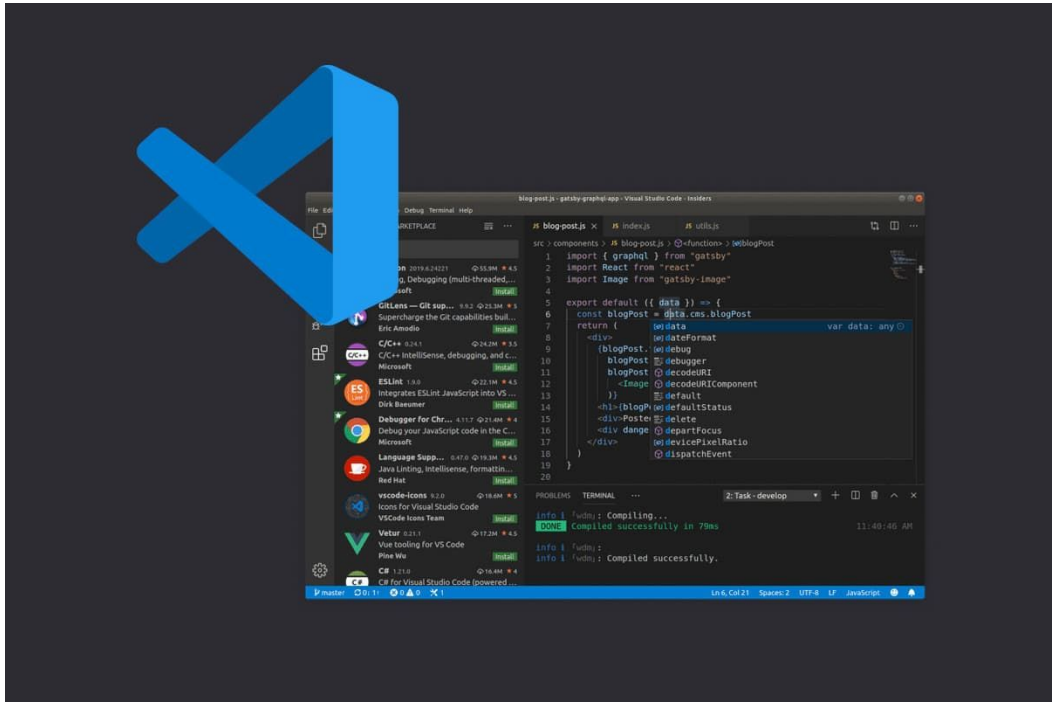
https://www.youtube.com/watch?v=7LCNuwTXiQw

Prompt sugerido:

a. *"Genera un archivo agents.md estricto para un proyecto en [Lenguaje]. Incluye reglas para: nombrado de variables en camelCase, uso obligatorio de interfaces, manejo de errores con try/catch y prohibición de usar 'any'.""*

FASE 1: Desarrollo Asistido por IA (Arquitectura y Contexto)

3. Generación de Boilerplate: Usen el IDE para generar la estructura inicial, conexiones a BD y configuraciones base.



FASE 11: Pair Programming con IA

1. **TDD Asistido:** Solicitar a la IA que genere los tests unitarios fallidos para la lógica de negocio principal (P. ej. validar que no se agende una cita con un veterinario ocupado)

Prompt sugerido:

a. *"Escribe 5 casos de prueba unitarios usando [Jest/PyTest/JUnit] para la función 'agendarCita'. Incluye casos de borde: veterinario ocupado, fecha en el pasado, y mascota no registrada. No escribas la implementación aún, solo los tests.'"*

FASE 11: Pair Programming con IA

2. Implementación Dirigida: Implementar la lógica para pasar los tests.

Si la IA sugiere código, deben leerlo y entenderlo.
Realizar informe con los resultados de los tests.

FASE 11: Pair Programming con IA

3. Refactorización: Una vez que el código funcione, usen la IA para mejorarlo.

Prompt sugerido:

a. *Revisa* este bloque de código. Identifica violaciones a los principios SOLID y refactorízalo para inyectar las dependencias correctamente. Explica los cambios.

FASE III: Flujo de Trabajo Humano-IA

1. **Code Review Automatizado:** Antes de hacer merge a la rama main, pidan a un LLM que actúe como un revisor de seguridad y rendimiento.

Prompt sugerido:

a. *Actúa como un experto en Ciberseguridad y Rendimiento. Revisa el siguiente código en busca de: 1) Vulnerabilidades (SQLi, XSS), 2) Fugas de memoria, 3) Queries ineficientes (N+1). Sé crítico y sugiere correcciones.*

FASE III: Flujo de Trabajo Humano-IA

2. **Automatización (CI/CD)**: Generar con la IA un archivo de GitHub Actions (o similar) para que corra los tests y un linter automáticamente en cada Push.

Prompt sugerido:

a. *Genera el YAML para un GitHub Action que se ejecute en cada push a 'main'. Debe instalar dependencias, correr el linter y ejecutar los tests unitarios. Falla si el coverage es menor al 80%.*

FASE III: Flujo de Trabajo Humano-IA

3. Documentación Autónoma: Generen el README.md y la documentación de la API.

Prompt sugerido:

a. *Actúa* como un Ingeniero de Software Principal y Arquitecto de Soluciones experto en documentación técnica. Tu objetivo es generar la documentación completa y profesional de un proyecto basándote en el código fuente, la estructura de archivos y los detalles técnicos que te proporcionaré a continuación.

b. Debes generar dos entregables principales en bloques de código independientes:
   i. El archivo `README.md` principal del proyecto.
   ii. La documentación técnica detallada de la API.

c. ### CONTEXTO DEL PROYECTO…